Chapter 7 ▪ Exceptions, Assertions, and Logging

In a perfect world, users would never enter data in the wrong form, files they choose to open would always exist, and code would never have bugs. So far, I've mostly presented code as if we all lived in this kind of perfect world. It is now time to turn to the mechanisms the Java programming language has for dealing with the real world of bad data and buggy code.

Encountering errors is unpleasant. If a user loses all the work he or she did during a program session because of a programming mistake or some external circumstance, that user may forever turn away from your program. At the very least, you must:

- Notify the user of an error;
- Save all work; and
- Allow users to gracefully exit the program.

For exceptional situations, such as bad input data with the potential to bomb the program, Java uses a form of error trapping called, naturally enough, *exception handling*. Exception handling in Java is similar to that in C++ or Delphi. The first part of this chapter covers Java's exceptions.

During testing, you need to run lots of checks to make sure your program does the right thing. But those checks can be time-consuming and unnecessary after testing has completed. You could just remove the checks and stick them back in when additional testing is required—but that is tedious. The second part of this chapter shows you how to use the assertion facility for selectively activating checks.

When your program does the wrong thing, you can't always communicate with the user or terminate. Instead, you may want to record the problem for later analysis. The third part of this chapter discusses the standard Java logging framework.

7.1. Dealing with Errors

Suppose an error occurs while a Java program is running. The error might be caused by a file containing wrong information, a flaky network connection, or (I hate to mention it) use of an invalid array index or an object reference that hasn't yet been assigned to an object. Users expect that programs will act sensibly when errors happen. If an operation cannot be completed because of an error, the program ought to either

- Return to a safe state and enable the user to execute other commands; or
- Allow the user to save all work and terminate the program gracefully.

This may not be easy to do, because the code that detects the error condition is usually far away from the code that can roll back the data to a safe state or save the user's work and exit cheerfully. The mission of exception handling is to transfer control from where the error occurred to an error handler that can deal with the situation. To handle exceptional situations in your program, you must take into account the errors and problems that may occur. What sorts of problems do you need to consider?

- *User input errors.* In addition to the inevitable typos, some users like to blaze their own trail instead of following directions. Suppose, for example, that a user asks to connect to a URL that is syntactically wrong. Your code should check the syntax, but suppose it does not. Then the network layer will complain.
- *Device errors.* Hardware does not always do what you want it to. The printer may be turned off. A web page may be temporarily unavailable. Devices will often fail in the middle of a task. For example, a printer may run out of paper during printing.
- *Physical limitations.* Disks can fill up; you can run out of available memory.
- *Code errors.* A method may not perform correctly. For example, it could deliver wrong answers or use other methods incorrectly. Computing an invalid array index, trying to find a nonexistent entry in a hash table, or trying to pop an empty stack are all examples of a code error.

The traditional reaction to an error in a method is to return a special error code that the calling method analyzes. For example, methods that read information back from files often return a `-1` end-of-file value marker rather than a standard character. Another common return value to denote an error condition is the `null` reference.

However, it is not always possible to return an error code. There may be no obvious way of distinguishing valid and invalid data. A method returning an integer cannot simply return `-1` to denote the error; the value `-1` might be a perfectly valid result.

Returning `null` to indicate failure is particularly problematic. Consider the `listFiles` method of the legacy `java.io.File` class. It returns a `File[]` array, or `null` in case of failure. If the programmer did not check for `null`, a `NullPointerException` occurs when the return value is first used, masking the original problem.

Java allows every method an alternative exit path if it is unable to complete its task in the normal way. In this situation, the method does not return a value. Instead, it *throws* an object that encapsulates the error information. Note that the method exits immediately; it does not return its normal (or any) value. Moreover, execution does not resume at the code that called the method; instead, the excep-

tion-handling mechanism begins its search for an *exception handler* that can deal with this particular error condition.

Exceptions have their own syntax and are part of a special inheritance hierarchy. I'll take up the syntax first and then give a few hints on how to use this language feature effectively.



Tip: Favor methods that use exceptions over those that return error codes. It is easy to forget checking error codes, but exceptions cannot be overlooked. Exceptions can also carry more information. For example, the `delete` method of the legacy `java.io.File` class returns `false` if the deletion was unsuccessful. The `delete` method in the `java.nio.file.Files` class is more useful. In case of failure it throws an exception of a type that clearly indicates the nature of the problem.

7.1.1. The Classification of Exceptions

In the Java programming language, an exception object is always an instance of a class derived from `Throwable`. As you will soon see, you can create your own exception classes if those built into Java do not suit your needs.

Figure 7.1 is a simplified diagram of the exception hierarchy in Java.

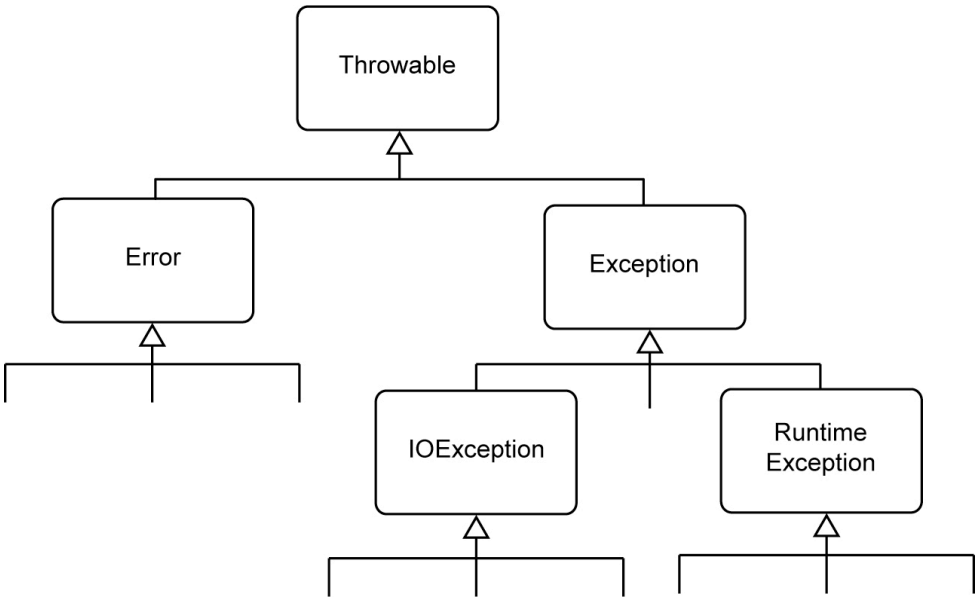


Figure 7.1: Exception hierarchy in Java

Notice that all exceptions descend from `Throwable`, but the hierarchy immediately splits into two branches: `Error` and `Exception`.

The `Error` hierarchy describes internal errors and resource exhaustion situations inside the Java runtime system. You should not throw an object of this type. There is little you can do if such an internal error occurs, beyond notifying the user and trying to terminate the program gracefully. These situations are quite rare.

When doing Java programming, focus on the `Exception` hierarchy. The `Exception` hierarchy also splits into two branches: exceptions that derive from `RuntimeException` and those that do not. The general rule is this: A `RuntimeException` happens because you made a programming error. Any other exception occurs because a bad thing, such as an I/O error, happened to your otherwise good program.

Exceptions that inherit from `RuntimeException` include such problems as

- A bad cast
- An out-of-bounds array access
- A null pointer access

Exceptions that do not inherit from `RuntimeException` include

- Trying to read past the end of a file
- Trying to open a file that doesn't exist
- Trying to find a `Class` object for a string that does not denote an existing class

The rule "If it is a `RuntimeException`, it was your fault" works pretty well. You could have avoided that `ArrayIndexOutOfBoundsException` by testing the array index against the array bounds. The `NullPointerException` would not have happened had you checked whether the variable was `null` before using it.

How about a file that doesn't exist? Can't you first check whether the file exists, and then open it? Well, the file might be deleted right after the check for its existence. Thus, the notion of "existence" depends on the environment, not just on your code.

The Java Language Specification calls any exception that derives from the class `Error` or the class `RuntimeException` an *unchecked* exception. All other exceptions are called *checked* exceptions. This is useful terminology that I also adopt in this book. The compiler checks that you provide exception handlers for all checked exceptions.



Note: The name `RuntimeException` is somewhat confusing. Of course, all of the errors we are discussing occur at runtime. The name originated in prehistoric times, when the "runtime" of Oak, the predecessor of Java, generated out-of-bounds exceptions and null pointer exceptions. Presumably I/O exceptions were produced by some other component.



Note: If you are familiar with the (much more limited) exception hierarchy of the standard C++ library, you may be really confused at this point. C++ has two fundamental exception classes, `runtime_error` and `logic_error`. The `logic_error` class is the equivalent of Java's `RuntimeException` and also denotes logical errors in the program. The `runtime_error` class is the superclass for exceptions caused by unpre-

dictable problems. It is equivalent to those exceptions in Java that are *not* of type `RuntimeException`.

7.1.2. Declaring Checked Exceptions

A Java method can throw an exception if it encounters a situation it cannot handle. The idea is simple: A method will not only tell the Java compiler what values it can return, *it is also going to tell the compiler what can go wrong*. For example, code that attempts to read from a file knows that the file might not exist or that it might be empty. The code that tries to process the information in a file therefore will need to notify the compiler that it can throw some sort of `IOException`.

The place in which you advertise that your method can throw an exception is the header of the method; the header changes to reflect the checked exceptions the method can throw. For example, here is the declaration of one of the constructors of the `FileInputStream` class from the standard library. (See **Chapter 2 of Volume II** for more on input and output.)

```
public FileInputStream(String name) throws FileNotFoundException
```

The declaration says that this constructor produces a `FileInputStream` object from a `String` parameter but that it *also* can go wrong in a special way—by throwing a `FileNotFoundException`. If this sad state should come to pass, the constructor call will not initialize a new `FileInputStream` object but instead will throw an object of the `FileNotFoundException` class. If it does, the runtime system will begin to search for an exception handler that knows how to deal with `FileNotFoundException` objects.

When you write your own methods, you don't have to advertise every possible throwable object that your method might actually throw. To understand when (and what) you have to advertise in the `throws` clause of the methods you write, keep in mind that an exception is thrown in any of the following four situations:

- You call a method that throws a checked exception—for example, the `FileInputStream` constructor.
- You detect an error and throw a checked exception with the `throw` statement (the `throw` statement is covered in the next section).
- You make a programming error, such as `a[-1] = 0` that gives rise to an unchecked exception (in this case, an `ArrayIndexOutOfBoundsException`).
- An internal error occurs in the virtual machine or runtime library.

If either of the first two scenarios occurs, you must tell the programmers who will use your method about the possibility of an exception. Why? Any method that throws an exception is a potential death trap. If no handler catches the exception, the current thread of execution terminates.

As with Java methods that are part of the supplied classes, you declare that your method may throw an exception with an *exception specification* in the method header.

```
class MyAnimation {
    . . .
    public Image loadImage(String s) throws IOException {
        . . .
    }
}
```

If a method might throw more than one checked exception type, you must list all exception classes in the header. Separate them by commas, as in the following example:

```
class MyAnimation {
    . . .
    public Image loadImage(String s) throws FileNotFoundException, EOFException {
        . . .
    }
}
```

However, you do not need to advertise internal Java errors—that is, exceptions inheriting from `Error`. Any code could potentially throw those exceptions, and they are entirely beyond your control.

Similarly, you should not advertise unchecked exceptions inheriting from `RuntimeException`.

```
class MyAnimation {
    . . .
    void drawImage(int i) throws ArrayIndexOutOfBoundsException { // bad style
        . . .
    }
}
```

These runtime errors are completely under your control. If you are so concerned about array index errors, you should spend your time fixing them instead of advertising the possibility that they can happen.

In summary, a method must declare all the *checked* exceptions that it might throw. Unchecked exceptions are either beyond your control (`Error`) or result from conditions that you should not have allowed in the first place (`RuntimeException`). If your method fails to faithfully declare all checked exceptions, the compiler will issue an error message.

Of course, as you have already seen in quite a few examples, instead of declaring the exception, you can also catch it. Then the exception won't be thrown out of the method, and no `throws` specification is necessary. You will see later in this chapter how to decide whether to catch an exception or to enable someone else to catch it.



Caution: If you override a method from a superclass, the checked exceptions that the subclass method declares cannot be more general than those of the superclass method. (It is OK to throw more specific exceptions, or not to throw any exceptions in the subclass method.) In particular, if the superclass method throws no checked exception at all, neither can the subclass. For example, if you override the `toString` method, you must not throw any checked exceptions, because the superclass method doesn't throw any.

When a method declares that it might throw an exception of a particular class, it may throw an instance of any of its subclasses. For example, suppose the `loadImage` method declares that it throws an `IOException`. In that case, you do not know what kind of `IOException` it is; it could be a plain `IOException` or an object of one of the various subclasses, such as `FileNotFoundException`.



Note: The `throws` specifier is the same as the `throw` specifier in C++, with one important difference. In C++, `throw` specifiers are enforced at run-time, not at compile time. That is, the C++ compiler pays no attention to exception specifications. But if an exception is thrown in a function that is not part of the `throw` list, the `unexpected` function is called, and, by default, the program terminates.

Also, in C++, a function may throw any exception if no `throw` specification is given. In Java, a method without a `throws` specifier may not throw any checked exceptions at all.

7.1.3. How to Throw an Exception

Now, suppose something terrible has happened in your code. You have a method, `readData`, that is reading in a file whose header promised

```
Content-length: 1024
```

but you got an end of file after 733 characters. You may decide this situation is so abnormal that you want to throw an exception.

You need to decide what exception type to throw. Some kind of `IOException` would be a good choice. Perusing the Java API documentation, you find an `EOFException` with the description "Signals that an EOF has been reached unexpectedly during input." Perfect. Here is how you throw it:

```
throw new EOFException();
```

or, if you prefer,

```
var e = new EOFException();
throw e;
```

Here is how it all fits together:

```
String readData(Scanner in) throws EOFException {
    . . .
    while (. . .) {
        if (!in.hasNextLine()) { // EOF encountered
            if (n < len)
                throw new EOFException();
        }
        . . .
    }
    return s;
}
```

The `EOFException` has a second constructor that takes a string argument. You can put this to good use by describing the exceptional condition more carefully.

```
String gripe = "Content-length: " + len + ", Received: " + n;
throw new EOFException(gripe);
```

As you can see, throwing an exception is easy if one of the existing exception classes works for you. In this case:

1. Find an appropriate exception class.
2. Make an object of that class.
3. Throw it.

Once a method throws an exception, it does not return to its caller. This means you do not have to worry about cooking up a default return value or an error code.



Note: Throwing an exception is the same in C++ and in Java, with one small difference. In Java, you can throw only objects of subclasses of `Throwable`. In C++, you can throw values of any type.

7.1.4. Creating Exception Classes

Your code may run into a problem which is not adequately described by any of the standard exception classes. In this case, it is easy enough to create your own exception class. Just derive it from `Exception`, or from a child class of `Exception` such as `IOException`. It is customary to give both a default constructor and a constructor that contains a detailed message. (The `toString` method of the `Throwable` superclass returns a string containing that detailed message, which is handy for debugging.)

```
class FileFormatException extends IOException {
    public FileFormatException() {}
    public FileFormatException(String gripe) {
```

```
        super(gripe);
    }
}
```

Now you are ready to throw your very own exception type.

```
String readData(Scanner in) throws FileFormatException {
    . . .
    while (. . .) {
        if (!in.hasNextLine()) { // EOF encountered
            if (n < len)
                throw new FileFormatException();
        }
        . . .
    }
    return s;
}
```

`java.lang.Throwable` 1.0

- `Throwable()`
  constructs a new `Throwable` object with no detailed message.
- `Throwable(String message)`
  constructs a new `Throwable` object with the specified detailed message. By convention, all derived exception classes support both a default constructor and a constructor with a detailed message.
- `String getMessage()`
  gets the detailed message of the `Throwable` object.

7.2. Catching Exceptions

You now know how to throw an exception. It is pretty easy: You throw it and you forget it. Of course, some code has to catch the exception. Catching exceptions requires more planning. That's what the next sections will cover.

7.2.1. Catching an Exception

If an exception occurs that is not caught anywhere, the program will terminate and print a message to the console, giving the type of the exception and a stack trace. However, GUI programs may catch exceptions, print stack trace messages, and then go back to the user interface processing loop. (When you are debugging a GUI program, it is a good idea to keep the console on the screen and not minimized.)

To catch an exception, set up a `try`/`catch` block. The simplest form of the `try` block is as follows:

```
try {
    code
    more code
    more code
}
catch (ExceptionType e) {
    handler for this type
}
```

Suppose any code inside the `try` block throws an exception of the class specified in the `catch` clause, or its subclass. In that case:

1. The program skips the remainder of the code in the `try` block.
2. The program executes the handler code inside the `catch` clause.

If none of the code inside the `try` block throws an exception, then the program skips the `catch` clause.

If any of the code in a method throws an exception of a type other than the one named in the `catch` clause, this method exits immediately. (Hopefully, one of its callers has already provided a `catch` clause for that type.)

To show this at work, here's some fairly typical code for reading in data:

```
public void read(String filename) {
    try {
        var in = new FileInputStream(filename);
        int b;
        while ((b = in.read()) != -1) {
            process input
        }
    }
    catch (IOException exception) {
        exception.printStackTrace();
    }
}
```

Notice that most of the code in the `try` clause is straightforward: It reads and processes bytes until it encounters the end of the file. As you can see by looking at the Java API, there is the possibility that the `read` method will throw an `IOException`. In that case, we skip out of the entire `while` loop, enter the `catch` clause, and generate a stack trace. For a toy program, that seems like a reasonable way to deal with this exception. What other choice do you have?

Often, the best choice is to do nothing at all and simply pass the exception on to the caller. If an error occurs in the `read` method, let the caller of the `read` method worry about it! If we take that approach, then we have to advertise the fact that the method may throw an `IOException`.

```
public void read(String filename) throws IOException {
    var in = new FileInputStream(filename);
    int b;
    while ((b = in.read()) != -1) {
        process input
    }
}
```

Remember, the compiler strictly enforces the `throws` specifiers. If you call a method that throws a checked exception, you must either handle it or pass it on.

Which of the two is better? As a general rule, you should catch those exceptions that you know how to handle and propagate those that you do not know how to handle.

When you propagate an exception, you must add a `throws` specifier to alert the caller that an exception may be thrown.

Look at the Java API documentation to see what methods throw which exceptions. Then decide whether you should handle them or add them to the `throws` list. There is nothing embarrassing about the latter choice. It is better to direct an exception to a competent handler than to squelch it.

However, when overriding a method, you are not allowed to add more `throws` specifiers to a subclass method than are present in the superclass method. Then you *must* catch the undeclared checked exceptions. In particular, if you override a method which throws no exceptions (such as `compareTo` in `Comparable`), then you have no choice but to catch each checked exception.

Sometimes, you do not need the exception object in a `catch` clause. Starting in Java 25, you can use an underscore as an unnamed variable:

```
catch (FileNotFoundException _) {
    System.exit(0);
}
```

7.2.2. Catching Multiple Exceptions

You can catch multiple exception types in a `try` block and handle each type differently. Use a separate `catch` clause for each type, as in the following example:

```
try {
    code that might throw exceptions
}
catch (FileNotFoundException e) {
    emergency action for missing files
}
catch (UnknownHostException e) {
    emergency action for unknown hosts
```

```
}
catch (IOException e) {
    emergency action for all other I/O problems
}
```

You need to order the `catch` clauses so that the more specific exception types come before the more general ones.

The exception object may contain information about the nature of the exception. To find out more about the object, try

```
e.getMessage()
```

to get the detailed error message (if there is one), or

```
e.getClass().getName()
```

to get the actual type of the exception object.

You can catch multiple exception types in the same `catch` clause. For example, suppose that the action for missing files and unknown hosts is the same. Then you can combine the `catch` clauses:

```
try {
    code that might throw exceptions
}
catch (FileNotFoundException | UnknownHostException e) {
    emergency action for missing files and unknown hosts
}
catch (IOException e) {
    emergency action for all other I/O problems
}
```

This feature is only needed when catching exception types that are not subclasses of one another.

When you catch multiple exceptions, the exception variable is implicitly `final`. For example, you cannot assign a different value to `e` in the body of the clause

```
catch (FileNotFoundException | UnknownHostException e) { . . . }
```

Catching multiple exceptions doesn't just make your code look simpler but also more efficient. The generated bytecodes contain a single block for the shared `catch` clause.



Caution: The type of the variable `e` is *not* the "union type" `FileNotFoundException | UnknownHostException`, even if your IDE

reports it as such. Java variables cannot have such union types. The actual type of `e` is the least upper bound of those types; in this case, `IOException`.

In most cases, you don't care. But it matters if you analyze `e` with a `switch`. The obvious `switch` has a problem—it is not exhaustive:

```
catch (FileNotFoundException | UnknownHostException e) {
    switch (e) { // ERROR: not exhaustive
        case FileNotFoundException fe -> . . .;
        case UnknownHostException ue -> . . .;
    }
    . . .
}
```

You can fix the problem by adding a `default` clause. However, it might be clearer to reorganize the code and use separate `catch` clauses.



Caution: Catch clauses do not pay attention to sealed exception hierarchies. Suppose you provide

```
sealed abstract class DomainException extends Exception
    permits FatalDomainException, SurvivableDomainException {}
```

Then the `catch` clauses

```
try {
    domainMethod(); // throws DomainException
}
catch (FatalDomainException e) { . . . }
catch (SurvivableDomainException e) { . . . }
```

are not sufficient, even though there can be no other checked exception instances.

7.2.3. Rethrowing and Chaining Exceptions

You can throw an exception in a `catch` clause. Typically, you do this when you want to change the exception type. If you build a subsystem that other programmers use, it makes a lot of sense to use an exception type that indicates a failure of the subsystem. An example of such an exception type is the `ServletException`. The code that executes a servlet may not want to know in minute detail what went wrong, but it definitely wants to know that the servlet was at fault.

Here is how you can catch an exception and rethrow it:

```
try {
    access the database
}
catch (SQLException e) {
    throw new ServletException("database error: " + e.getMessage());
}
```

Here, the `ServletException` is constructed with the message text of the exception.

However, it is a better idea to set the original exception as the “cause” of the new exception:

```
try {
    access the database
}
catch (SQLException original) {
    var e = new ServletException("database error", original);
    throw e;
}
```

The `ServletException` class has a constructor to which you can pass the cause. If you use an exception type for which that is not the case, you can instead call

```
e.initCause(original);
```

When the exception is caught, the original exception can be retrieved:

```
Throwable original = caughtException.getCause();
```

This wrapping technique is highly recommended. It allows you to throw high-level exceptions in subsystems without losing the details of the original failure.



Tip: The wrapping technique is also useful if a checked exception occurs in a method that is not allowed to throw a checked exception. You can catch the checked exception and wrap it into a runtime exception.

As this situation often arises with input/output, the Java API provides an `UncheckedIOException` that you can use as a wrapper for the checked `IOException`.

Sometimes, you just want to log an exception and rethrow it without any change:

```
try {
    access the database
}
catch (Exception e) {
```

```
    logger.log(level, message, e);
    throw e;
}
```

There is a subtle point here. Suppose the code is inside a method

```
public void updateRecord() throws SQLException
```

The Java compiler tracks the fact that `e` originates from the `try` block. Provided that the only checked exceptions in that block are `SQLException` instances, and provided that `e` is not changed in the `catch` block, it is valid to declare the enclosing method as `throws SQLException`.

7.2.4. The `finally` Clause

When your code throws an exception, it stops processing the remaining code in your method and exits the method. This is a problem if the method has acquired some local resource, which only this method knows about, and that resource must be cleaned up. One solution is to catch all exceptions, carry out the cleanup, and rethrow the exceptions. But this solution is tedious because you need to clean up the resource allocation in two places—in the normal code and in the exception code. The `finally` clause can solve this problem.



Note: There is a more elegant solution, the `try`-with-resources statement that you will see in the following section. We discuss the `finally` mechanism in detail because it is the conceptual foundation. But in practice, you will use `try`-with-resources statements far more often than `finally` clauses.

The code in the `finally` clause executes whether or not an exception was caught. In the following example, the program will close the input stream *under all circumstances*:

```
var in = new FileInputStream(. . .);
try {
    // 1
    code that might throw exceptions
    // 2
}
catch (IOException e) {
    // 3
    show error message
    // 4
}
finally {
    // 5
    in.close();
}
// 6
```

Let us look at the three possible situations in which the program will execute the `finally` clause.

1. The code throws no exceptions. In this case, the program first executes all the code in the `try` block. Then, it executes the code in the `finally` clause. Afterwards, execution continues with the first statement after the `finally` clause. In other words, execution passes through points 1, 2, 5, and 6.
2. The code throws an exception that is caught in a `catch` clause—in our case, an `IOException`. For this, the program executes all code in the `try` block, up to the point at which the exception was thrown. The remaining code in the `try` block is skipped. The program then executes the code in the matching `catch` clause, and then the code in the `finally` clause.
   If the `catch` clause does not throw an exception, the program executes the first line after the `finally` clause. In this scenario, execution passes through points 1, 3, 4, 5, and 6.
   If the `catch` clause throws an exception, then the exception is thrown back to the caller of this method, and execution passes through points 1, 3, and 5 only.
3. The code throws an exception that is not caught in any `catch` clause. Here, the program executes all code in the `try` block until the exception is thrown. The remaining code in the `try` block is skipped. Then, the code in the `finally` clause is executed, and the exception is thrown back to the caller of this method. Execution passes through points 1 and 5 only.

You can use the `finally` clause without a `catch` clause. For example, consider the following `try` statement:

```
InputStream in = . . .;
try {
    code that might throw exceptions
}
finally {
    in.close();
}
```

The `in.close()` statement in the `finally` clause is executed whether or not an exception is encountered in the `try` block. Of course, if an exception is encountered, it is rethrown and must be caught in another `catch` clause.

```
InputStream in = . . .;
try {
    try {
        code that might throw exceptions
    }
    finally {
        in.close();
    }
}
catch (IOException e) {
    show error message
}
```

The inner `try` block has a single responsibility: to make sure that the input stream is closed. The outer `try` block has a single responsibility: to ensure that errors are reported. Not only is this solution clearer, it is also more functional: Errors in the `finally` clause are reported.



Caution: A `finally` clause can yield unexpected results when it contains `return` statements. Suppose you exit the middle of a `try` block with a `return` statement. Before the method returns, the `finally` block is executed. If the `finally` block also contains a `return` statement, then it masks the original return value. Consider this example:

```
public static int parseInt(String s) {
    try {
        return Integer.parseInt(s);
    }
    finally {
        return 0; // ERROR
    }
}
```

It looks as if in the call `parseInt("42")`, the body of the `try` block returns the integer `42`. However, the `finally` clause is executed before the method actually returns and causes the method to return `0`, ignoring the original return value.

And it gets worse. Consider the call `parseInt("zero")`. The `Integer.parseInt` method throws a `NumberFormatException`. Then the `finally` clause is executed, and the `return` statement swallows the exception!

The body of the finally clause is intended for cleaning up resources. Don't put statements that change the control flow (`return`, `throw`, `break`, `continue`) inside a `finally` clause.

7.2.5. The `try`-with-Resources Statement

There is a useful shortcut to the code pattern

```
open a resource
try {
    work with the resource
}
finally {
    close the resource
}
```

provided the resource belongs to a class that implements the `AutoCloseable` interface. That interface has a single method

```
void close() throws Exception
```



Note: There is also a `Closeable` interface. It is a subinterface of `AutoCloseable`, also with a single `close` method. However, that method is declared to throw an `IOException`.

In its simplest variant, the `try`-with-resources statement has the form

```
try (Resource res = . . .) {
    work with res
}
```

When the `try` block exits, then `res.close()` is called automatically. Here is a typical example—processing all lines of a file:

```
try (var in = new Scanner(Path.of("in.txt"))) {
    while (in.hasNextLine()) {
        String line = in.nextLine();
        // Process line
    }
}
```

When the block exits normally, or when there was an exception, the `in.close()` method is called, exactly as if you had used a `finally` block.

You can specify multiple resources. For example,

```
try (var in = new Scanner(Path.of("in.txt"));
      var out = new PrintWriter("out.txt")) {
    while (in.hasLine()) {
        String line = in.nextLine();
        out.println(line.toUpperCase());
    }
}
```

No matter how the block exits, both `in` and `out` are closed. If you programmed this by hand, you would have needed two nested `try`/`finally` statements.

You can provide previously declared effectively final variables in the `try` header:

```
public static void printAll(String[] lines, PrintWriter out) {
    try (out) { // effectively final variable
        for (String line : lines)
            out.println(line);
    } // out.close() called here
}
```

This is not common, since one generally prefers to invoke `close` in the same scope in which the object was constructed.

A difficulty arises when the `try` block throws an exception and the `close` method also throws an exception. The `try`-with-resources statement handles this situation quite elegantly. The original exception is rethrown, and any exceptions thrown by `close` methods are considered "suppressed." They are automatically caught and added to the original exception with the `addSuppressed` method. If you are interested in them, call the `getSuppressed` method which yields an array of the suppressed expressions from `close` methods.

You don't want to program this by hand. Use the `try`-with-resources statement whenever you need to close a resource.



Note: A `try`-with-resources statement can itself have `catch` clauses and even a `finally` clause. These are executed after closing the resources.

7.2.6. Analyzing Stack Trace Elements

A *stack trace* is a listing of all pending method calls at a particular point in the execution of a program. You have almost certainly seen stack trace listings—they are displayed whenever a Java program terminates with an uncaught exception.

You can access the text description of a stack trace by calling the `printStackTrace` method of the `Throwable` class.

```
var t = new Throwable();
var out = new StringWriter();
t.printStackTrace(new PrintWriter(out));
String description = out.toString();
```

A more flexible approach is the `StackWalker` class that yields a stream of `StackWalker.StackFrame` instances, each describing one stack frame. You can iterate over the stack frames with this call:

```
StackWalker walker = StackWalker.getInstance();
walker.forEach(frame -> analyze frame)
```

If you want to process the `Stream<StackWalker.StackFrame>` lazily, call

```
walker.walk(stream -> process stream)
```

Stream processing is described in detail in **Chapter 1 of Volume II**.

The `StackWalker.StackFrame` class has methods to obtain the file name and line number, as well as the class object and method name, of the executing line of

code. The `toString` method yields a formatted string containing all of this information.



Note: The `Throwable.getStackTrace` method yields a `StackTraceElement[]` array with similar information as the stream of `StackWalker.StackFrame` instances. However, that call is less efficient since it captures the entire stack even though the caller may only need a few frames, and it only provides access to the class names, but not the class objects, of the pending methods.

Listing 7.1 prints the stack trace of a recursive factorial function. For example, if you compute `factorial(3)`, the printout is

```
factorial(3):
stackTrace.StackTraceDemo.factorial(StackTraceDemo.java:20)
stackTrace.StackTraceDemo.main(StackTraceDemo.java:36)
factorial(2):
stackTrace.StackTraceDemo.factorial(StackTraceDemo.java:20)
stackTrace.StackTraceDemo.factorial(StackTraceDemo.java:26)
stackTrace.StackTraceDemo.main(StackTraceDemo.java:36)
factorial(1):
stackTrace.StackTraceDemo.factorial(StackTraceDemo.java:20)
stackTrace.StackTraceDemo.factorial(StackTraceDemo.java:26)
stackTrace.StackTraceDemo.factorial(StackTraceDemo.java:26)
stackTrace.StackTraceDemo.main(StackTraceDemo.java:36)
return 1
return 2
return 6
```

Listing 7.1 v1ch07/StackTraceDemo.java

```
1  package v1ch07;
2
3  /**
4   * A program that displays a trace feature of a recursive method call.
5   */
6  class StackTraceDemo {
7      void main() {
8          int n = Integer.parseInt(IO.readln("Enter n: "));
9          factorial(n);
10     }
11
12     /**
13      * Computes the factorial of a number
14      * @param n a non-negative integer
15      * @return n! = 1 * 2 * . . . * n
16      */
17     int factorial(int n) {
18         IO.println("factorial(" + n + "):");
19         var walker = StackWalker.getInstance();
20         walker.forEach(IO::println);
21         int r;
22         if (n <= 1)
```

```
23              r = 1;
24          else
25              r = n * factorial(n - 1);
26          IO.println("return " + r);
27          return r;
28      }
29  }
```

`java.lang.Throwable` 1.0

- `Throwable(Throwable cause)` 1.4
- `Throwable(String message, Throwable cause)` 1.4
  construct a `Throwable` with a given cause.
- `Throwable initCause(Throwable cause)` 1.4
  sets the cause for this object or throws an exception if this object already has a cause. Returns `this`.
- `Throwable getCause()` 1.4
  gets the exception object that was set as the cause for this object, or `null` if no cause was set.
- `StackTraceElement[] getStackTrace()` 1.4
  gets the trace of the call stack at the time this object was constructed.
- `void addSuppressed(Throwable t)` 7
  adds a "suppressed" exception to this exception. This happens in a `try`-with-resources statement where `t` is an exception thrown by a `close` method.
- `Throwable[] getSuppressed()` 7
  gets all "suppressed" exceptions of this exception. Typically, these are exceptions thrown by a `close` method in a `try`-with-resources statement.

`java.lang.Exception` 1.0

- `Exception(Throwable cause)` 1.4
- `Exception(String message, Throwable cause)`
  construct an `Exception` with a given cause.

`java.lang.RuntimeException` 1.0

- `RuntimeException(Throwable cause)` 1.4
- `RuntimeException(String message, Throwable cause)` 1.4
  construct a `RuntimeException` with a given cause.

`java.lang.StackWalker` 9

- `static StackWalker getInstance()`
- `static StackWalker getInstance(StackWalker.Option option)`
- `static StackWalker getInstance(Set<StackWalker.Option> options)`
  get a `StackWalker` instance. The options include `RETAIN_CLASS_REFERENCE`, `SHOW_HIDDEN_FRAMES`, and `SHOW_REFLECT_FRAMES` from the `StackWalker.Option` enumeration.
- `forEach(Consumer<? super StackWalker.StackFrame> action)`
  carries out the given action on each stack frame, starting with the most recently called method.
- `walk(Function<? super Stream<StackWalker.StackFrame>,? extends T> function)`
  applies the given function to the stream of stack frames and returns the result of the function.

`java.lang.StackWalker.StackFrame` 9

- `String getFileName()`
  gets the name of the source file containing the execution point of this element, or `null` if the information is not available.
- `int getLineNumber()`
  gets the line number of the source file containing the execution point of this element, or `-1` if the information is not available.
- `String getClassName()`
  gets the qualified name of the class whose method contains the execution point of this element.
- `Class<?> getDeclaringClass()`
  gets the `Class` object of the method containing the execution point of this element. An exception is thrown if the stack walker was not constructed with the `RETAIN_CLASS_REFERENCE` option.
- `String getMethodName()`
  gets the name of the method containing the execution point of this element. The name of a constructor is `<init>`. The name of a static initializer is `<clinit>`. You can't distinguish between overloaded methods with the same name.
- `boolean isNativeMethod()`
  returns `true` if the execution point of this element is inside a native method.
- `String toString()`
  returns a formatted string containing the class and method name and the file name and line number, if available.

`java.lang.StackTraceElement` 1.4

- `String getFileName()`
  gets the name of the source file containing the execution point of this element, or `null` if the information is not available.
- `int getLineNumber()`
  gets the line number of the source file containing the execution point of this element, or `-1` if the information is not available.
- `String getClassName()`
  gets the qualified name of the class containing the execution point of this element.
- `String getMethodName()`
  gets the name of the method containing the execution point of this element. The name of a constructor is `<init>`. The name of a static initializer is `<clinit>`. You can't distinguish between overloaded methods with the same name.
- `boolean isNativeMethod()`
  returns `true` if the execution point of this element is inside a native method.
- `String toString()`
  returns a formatted string containing the class and method name and the file name and line number, if available.

7.3. Tips for Using Exceptions

There is a certain amount of controversy about the proper use of exceptions. Some programmers believe that all checked exceptions are a nuisance, others can't seem to throw enough of them. I think that exceptions (even checked exceptions) have their place, so I offer you these tips for their proper use.

1. *Exception handling is not supposed to replace a simple test.*
   As an example of this, here's code that tries 10,000,000 times to pop an empty stack. It first does this by finding out whether the stack is empty.

   ```
   if (!s.empty()) s.pop();
   ```

   Next, we force it to pop the stack no matter what and catch the `EmptyStackException` that tells us we should not have done that.

   ```
   try {
       s.pop();
   }
   catch (EmptyStackException e) {
   }
   ```

   On my test machine, the version that calls `isEmpty` ran in 0.167 seconds. The version that catches the `EmptyStackException` ran in 3.629 seconds.

As you can see, it took far longer to catch an exception than to perform a simple test. The moral is: Use exceptions for exceptional circumstances only.

2. *Do not micromanage exceptions.*
   Some programmers wrap every statement in a separate `try` block.

```
PrintStream out;
Stack s;

for (i = 0; i < 100; i++) {
    try {
        n = s.pop();
    }
    catch (EmptyStackException e) {
        // stack was empty
    }
    try {
        out.writeInt(n);
    }
    catch (IOException e) {
        // problem writing to file
    }
}
```

   This approach blows up your code dramatically. Think about the task that you want the code to accomplish. Here, we want to pop 100 numbers off a stack and save them to a file. (Never mind why—it is just a toy example.) There is nothing we can do if a problem rears its ugly head. If the stack is empty, it will not become occupied. If the file contains an error, the error will not magically go away. It therefore makes sense to wrap the *entire task* in a `try` block. If any one operation fails, you can then abandon the task.

```
try {
    for (i = 0; i < 100; i++) {
        n = s.pop();
        out.writeInt(n);
    }
}
catch (IOException e) {
    // problem writing to file
}
catch (EmptyStackException e) {
    // stack was empty
}
```

   This code looks much cleaner. It fulfills one of the promises of exception handling: to *separate* normal processing from error handling.

3. *Make good use of the exception hierarchy.*
   Don't just throw an `Exception` or `RuntimeException`. Find an appropriate subclass or create your own.

Don't just catch `Exception`. It makes your code hard to read and maintain. Catch only those exception classes that you expect. And don't catch `Throwable`. You might hide an `OutOfMemoryError`.

Respect the difference between checked and unchecked exceptions. Checked exceptions are inherently burdensome—don't throw them for logic errors. (For example, the reflection library gets this wrong. Callers often need to catch exceptions that they know can never happen.)

Do not hesitate to turn an exception into another exception that is more appropriate. For example, when you parse an integer in a file, catch the `NumberFormatException` and set it as the cause of a subclass of `IOException` or `MySubsystemException`.

4. *Do not squelch exceptions.*

   In Java, there is a tremendous temptation to shut up exceptions. If you're writing a method that calls a method that might throw an exception once a century, the compiler whines because you have not declared the exception in the `throws` list of your method. You do not want to put it in the `throws` list because then the compiler will whine about all the methods that call your method. So you just shut it up:

   ```
   public Image loadImage(String s) {
       try {
            code that threatens to throw checked exceptions
       }
       catch (Exception e) {
       } // so there
   }
   ```

   Now your code will compile without a hitch. It will run fine, except when an exception occurs. Then, the exception will be silently ignored. If you believe that exceptions are at all important, you should make some effort to handle them right.

5. *When you detect an error, "tough love" works better than indulgence.*

   Some programmers worry about throwing exceptions when they detect errors. Maybe it would be better to return a dummy value rather than throw an exception when a method is called with invalid arguments? For example, should `Stack.pop` return `null`, or throw an exception when a stack is empty? I think it is better to throw an `EmptyStackException` at the point of failure than to have a `NullPointerException` occur at later time.

6. *Propagating exceptions is not a sign of shame.*

   Some programmers feel compelled to catch all exceptions that are thrown. If they call a method that throws an exception, such as the `FileInputStream` constructor or the `readLine` method, they instinctively catch the exception that may be generated. In many situations, it is actually better to *propagate* the exception instead of catching it:

   ```
   public void readStuff(String filename) throws IOException { // not a sign of shame!
       var in = new FileInputStream(filename);
       . . .
   ```

```
}
```

Higher-level methods are often better equipped to inform the user of errors or to abandon unsuccessful commands.

7. *Use standard methods for reporting null-pointer and out-of-bounds exceptions.*
   The `Objects` class has methods

```
requireNonNull
checkIndex
checkFromToIndex
checkFromIndexSize
```

   for these common checks. Use them for parameter validation:

```
public void putData(int position, Object newValue) {
    Objects.checkIndex(position, data.length);
    Objects.requireNonNull(newValue);
    . . .
}
```

   If the method is called with an invalid index or a null argument, an exception is thrown, using the familiar message that the Java library uses.

8. *Don't show stack traces to end users.*
   If your program encounters an unexpected exception, it may seem a good idea to display the stack trace so the users can report it, making it easier for you to pinpoint the issue. However, stack traces can contain implementation details that you do not want to reveal to potential attackers, such as the versions of libraries that you are using.
   Log the stack trace so that you can retrieve it, but only display a summary message to your users.



Note: Rules 5 and 6 can be summarized as "throw early, catch late."

7.4. Using Assertions

Assertions are a commonly used idiom of defensive programming. In the following sections, you will learn how to use them effectively.

7.4.1. The Assertion Concept

Suppose you are convinced that a particular property is fulfilled, and you rely on that property in your code. For example, you may be computing

```
double y = Math.sqrt(x);
```

You are certain that `x` is not negative. Perhaps it is the result of another computation that can't have a negative result, or it is a parameter of a method that requires its callers to supply only positive inputs. Still, you want to double-check rather than allow confusing "not a number" floating-point values creep into your computation. You could, of course, throw an exception:

```
if (x < 0) throw new IllegalArgumentException("x < 0");
```

But this code stays in the program, even after testing is complete. If you have lots of checks of this kind, the program may run quite a bit slower than it should.

The assertion mechanism allows you to put in checks during testing and to have them automatically removed in the production code.

The Java language has a keyword `assert`. There are two forms:

```
assert condition;
```

and

```
assert condition : expression;
```

Both statements evaluate the condition and throw an `AssertionError` if it is `false`. In the second statement, the expression is passed to the constructor of the `AssertionError` object and turned into a message string.



Note: The sole purpose of the *expression* part is to produce a message string. The `AssertionError` object does not store the actual expression value, so you can't query it later. As the JDK documentation states, doing so "would encourage programmers to attempt to recover from assertion failure, which defeats the purpose of the facility."

To assert that `x` is non-negative, you can simply use the statement

```
assert x >= 0;
```

Or you can pass the actual value of `x` into the `AssertionError` object, so that it gets displayed later.

```
assert x >= 0 : x;
```



Note: The `assert` macro of the C language turns the assertion condition into a string that is printed if the assertion fails. For example, if `assert(x >= 0)` fails, it prints that `"x >= 0"` is the failing condition. In Java, the condition is not automatically part of the error report. If you want to see it,

you have to pass it as a string into the `AssertionError` object: `assert x >= 0 : "x >= 0"`.

7.4.2. Assertion Enabling and Disabling

By default, assertions are disabled. Enable them by running the program with the `-enableassertions` or `-ea` option:

```
java -enableassertions MyProgram
```

Note that you do not have to recompile your program to enable or disable assertions. Enabling or disabling assertions is a function of the *class loader*. When assertions are disabled, the class loader strips out the assertion code so that it won't slow execution.

You can even turn on assertions in specific classes or in entire packages. For example:

```
java -ea:MyClass -ea:com.mycompany.mypackage... MyProgram
```

This command turns on assertions for the class `MyClass` and all classes in the `com.mycompany.mypackage` package *and its subpackages*. The option `-ea:...` turns on assertions in all classes of the unnamed package.

You can also disable assertions in certain classes and packages with the `-disableassertions` or `-da` option:

```
java -ea:... -da:MyClass MyProgram
```

Some classes are not loaded by a class loader but directly by the virtual machine. You can use these switches to selectively enable or disable assertions in those classes.

However, the `-ea` and `-da` switches that enable or disable all assertions do not apply to the "system classes" without class loaders. Use the `-enablesystemassertions/-esa` switch to enable assertions in system classes.

It is also possible to programmatically control the assertion status of class loaders. See the API notes at the end of this section.



Caution: When assertions are turned off, the assertion condition does not execute. Therefore, it is important that the condition does not have a side effect. Here is an example of something that you should not do:

```
File file = new File(filename);
. . .
assert file.delete(); // BAD
```

The intent was to make sure that the `delete` method returns `true`, which means the file was properly deleted. But when assertions are turned off, the deletion won't happen!

The remedy is to move the side effect outside the `assert` statement:

```
boolean deletionSuccessful = file.delete();
assert deletionSuccessful;
```



Note: The source code for the Java library has over four hundred assertions that are commented out. Some programmers comment out assertions after testing because otherwise they take up space in the class files. If you are concerned about that, you can conditionally include them as follows:

```
public static final boolean ASSERTS = true; // Recompile with false for production
. . .
if (ASSERTS) assert x >= 0;
```

7.4.3. Using Assertions for Parameter Checking

The Java language gives you three mechanisms to deal with system failures:

- Throwing an exception
- Logging
- Using assertions

When should you choose assertions? Keep these points in mind:

- Assertion failures are intended to be fatal, unrecoverable errors.
- Assertion checks are turned on only during development and testing. (This is sometimes jokingly described as "wearing a life jacket when you are close to shore, and throwing it overboard once you are in the middle of the ocean.")

Therefore, you would not use assertions for signaling recoverable conditions to another part of the program or for communicating problems to the program user. Assertions should only be used to locate internal program errors during testing.

Let's look at a common scenario—the checking of method parameters. Should you use assertions to check for illegal index values or `null` references? To answer that question, you have to look at the documentation of the method. Suppose you implement a sorting method.

```
/**
   Sorts the specified range of the specified array in ascending numerical order.
   The range to be sorted extends from fromIndex, inclusive, to toIndex, exclusive.
   @param a the array to be sorted
   @param fromIndex the index of the first element (inclusive) to be sorted
   @param toIndex the index of the last element (exclusive) to be sorted
   @throws IllegalArgumentException if fromIndex > toIndex
```

```
   @throws ArrayIndexOutOfBoundsException if fromIndex < 0 or toIndex > a.length
*/
static void sort(int[] a, int fromIndex, int toIndex)
```

The documentation states that the method throws an exception if the index values are incorrect. That behavior is part of the contract that the method makes with its callers. If you implement the method, you have to respect that contract and throw the indicated exceptions. It would not be appropriate to use assertions instead.

Should you assert that `a` is not `null`? That is not appropriate either. The method documentation is silent on the behavior of the method when `a` is `null`. The callers have the right to assume that the method will return successfully in that case and not throw an assertion error.

However, suppose the method contract had been slightly different:

```
@param a the array to be sorted (must not be null)
```

Now the callers of the method have been put on notice that it is illegal to call the method with a `null` array. Then the method may start with the assertion

```
assert a != null;
```

Computer scientists call this kind of contract a *precondition*. The original method had no preconditions on its parameters—it promised a well-defined behavior in all cases. The revised method has a single precondition: that `a` is not `null`. If the caller fails to fulfill the precondition, then all bets are off and the method can do anything it wants. In fact, with the assertion in place, the method does just that. It sometimes throws an assertion error, and sometimes a null pointer exception, depending on how its class loader is configured.

7.4.4. Using Assertions for Documenting Assumptions

Often, programmers use comments to document their underlying assumptions. Consider this example from **https://docs.oracle.com/javase/8/docs/technotes/guides/language/assert.html**:

```
if (i % 3 == 0)
    . . .
else if (i % 3 == 1)
    . . .
else // (i % 3 == 2)
    . . .
```

In this case, it makes a lot of sense to use an assertion instead.

```
if (i % 3 == 0)
    . . .
else if (i % 3 == 1)
    . . .
else {
    assert i % 3 == 2;
   . . .
}
```

Of course, it would make even more sense to think through the issue thoroughly. What are the possible values of `i % 3`? If `i` is positive, the remainders must be `0`, `1`, or `2`. If `i` is negative, then the remainders can be `-1` or `-2`. Thus, the real assumption is that `i` is not negative. A better assertion would be

```
assert i >= 0;
```

before the `if` statement.

At any rate, this example shows a good use of assertions as a self-check for the programmer. As you can see, assertions are a tactical tool for testing and debugging. In contrast, logging is a strategic tool for the entire lifecycle of a program. We will examine logging in the next section.

`java.lang.ClassLoader` 1.0

- `void setDefaultAssertionStatus(boolean b)` 1.4
  enables or disables assertions for all classes loaded by this class loader that don't have an explicit class or package assertion status.
- `void setClassAssertionStatus(String className, boolean b)` 1.4
  enables or disables assertions for the given class and its inner classes.
- `void setPackageAssertionStatus(String packageName, boolean b)` 1.4
  enables or disables assertions for all classes in the given package and its subpackages.
- `void clearAssertionStatus()` 1.4
  removes all explicit class and package assertion status settings and disables assertions for all classes loaded by this class loader.

7.5. Logging

Every Java programmer is familiar with the process of inserting print statements into troublesome code to gain insight into program behavior. Of course, once you have figured out the cause of trouble, you remove the print statements—only to put them back in when the next problem surfaces. Logging frameworks are designed to overcome this problem.

7.5.1. Should You Use the Java Logging Framework?

Java has a standard logging framework, usually called after its package name `java.util.logging` and sometimes abbreviated as `j.u.l`. However, other logging frameworks have more features and are in common use, such as Log4j (**https://logging.apache.org/log4j/2.x**) and Logback (**https://logback.qos.ch**).

If you want to give users of your code the choice of logging framework, then you should use a "façade" library that sends log messages to the preferred framework. A commonly used façade with a pleasant API is SLF4J (**https://www.slf4j.org**). Another façade is the "platform logging API" (also known as JEP 264). It is very basic but a part of the JDK. The façade is sometimes called the *frontend*. It provides the API that programmers use to log messages. The *backend* is in charge of filtering and formatting the messages, and putting them somewhere. The backend needs to be configurable by deployers, usually by editing configuration files.

In the following sections, I will show you how to use the platform logging API as frontend and `java.util.logging` as a backend. This can be a reasonable choice if you find the frontend API sufficient, since you can always swap out the backend.

The `java.util.logging` backend has fewer features than its more popular alternatives, but it suffices for many use cases. Because of its simplicity, it is less susceptible to attacks. In contrast, obscure features of Log4j allowed hackers to craft program inputs that, when logged, caused malicious code execution.

Whether or not you end up using them, studying the platform logging API and `java.util.logging` backend gives you a good foundation of the capabilities of logging frameworks.



Note: The API and backend that the following sections describe are for *application logging*. You can also turn on logging for the virtual machine, for example to log garbage collection. Use the `-Xlog` command-line option when starting the VM. For example,

```
java -Xlog:gc=trace:file=gc.log:uptime,tid MyProgram
```

See **https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html#enable-logging-with-the-jvm-unified-logging-framework** for details.

7.5.2. Logging 101

Platform loggers implement the `System.Logger` interface. Each logger has a name. The name can be arbitrary, but it is often the package name of the class whose methods generate logging messages. You get a platform logger like this:

```
System.Logger logger = System.getLogger("com.mycompany.myapp");
```

When you request a logger with a given name for the first time, it is created. Subsequent calls to the same name yield the same logger object.

Now you are ready to log:

```
logger.log(System.Logger.Level.INFO, "Opening file " + filename);
```

The record is printed like this:

```
Aug 04, 2025 09:53:34 AM com.mycompany.myapp.Main read
INFO: Opening file data.txt
```

Note that the time and the names of the calling class and method are automatically included.

To turn off these informational messages when your program is deployed, you can configure the backend. In the case of the `java.util.logging` backend, prepare a file `logging.properties` with the following contents:

```
handlers=java.util.logging.ConsoleHandler
com.mycompany.myapp.level=WARNING
```

Then start the application like this:

```
java -Djava.util.logging.config.file=logging.properties com.mycompany.myapp.Main
```

Since the `INFO` level is below the `WARNING` level, the message no longer shows up.

The API for getting the logger and logging a message is part of the frontend—in this case, the platform logging API. If you use a different frontend, the API will be different.

The message destination, formatting, and filtering, as well as the mechanisms for the configuration are part of the backend—here, `java.util.logging`. If you use a different backend, follow its instructions for configuration.

7.5.3. The Platform Logging API

As you saw in the preceding section, each logged message has a level. The enumeration `System.Logger.Level` has the following values, in decreasing severity: `ERROR`, `WARNING`, `INFO`, `DEBUG`, and `TRACE`.



Tip: With the import statement

```
import static java.lang.System.Logger.Level.*;
```

you can shorten the levels:

```
logger.log(INFO, "Opening file " + filename);
    // Instead of System.Logger.Level.INFO
```

In the above example, the message `"Opening file " + filename` is created even if the message is suppressed. If you are concerned with the cost of creating the message string, you can use a lambda expression instead:

```
logger.log(INFO, () -> "Opening file " + filename);
```

Then the message is only computed when it is actually logged.

It is common to log an exception. The log includes the stack trace.

```
catch (IOException ex) {
    logger.log(WARNING, "Cannot open file " + filename, ex);
}
```

You can format the message using a pattern:

```
logger.log(WARNING, "Cannot open file {0}", filename);
```



Caution: This is *not* a `printf` style pattern. The pattern is processed by the `MessageFormat` class that is used for localization of program messages (see Volume II for details). Here, you just need to know that string placeholders are `{0}`, `{1}`, and so on. Braces must be escaped with single quotes: `'{'`, `'}'`. Use two single quotes `''` to include a literal single quote in the message.

Log messages can be localized to different languages, using the resource bundle mechanism that is introduced in Volume II.

Supply the bundle and the key for the formatting string:

```
logger.log(WARNING, bundle, "file.bad", filename);
   // Looks up file.bad in the bundle
```

Alternatively, get the logger as

```
System.Logger logger = System.getLogger("com.mycompany.myapp", bundle);
```

When calling one of the logger methods with a `String` parameter for the message or format, the argument is interpreted as a key in the bundle. The methods

with `Object` or `Supplier<String>` parameters are unaffected.

Some, but not all combinations of these features (deferred message computation, adding a throwable, formatting, using a bundle) are supported. See the API notes for details.

7.5.4. Logging Configuration

Let us now turn to the logging backend. As already mentioned, the default backend of the platform logging API is `java.util.logging`. The information in the following sections is specific to that backend.

You can change various properties of the backend by editing a configuration file. The default configuration file is located at `conf/logging.properties` in the JDK. To use another file, set the `java.util.logging.config.file` property to the file location by starting your application with

```
java -Djava.util.logging.config.file=configFile MainClass
```



Caution: Calling `System.setProperty("java.util.logging.config.file", configFile)` in `main` has no effect because the log manager is initialized during VM startup, before `main` executes.

You can specify the logging levels for your own loggers by adding lines such as

```
com.mycompany.myapp.level=WARNING
```

That is, append the `.level` suffix to the logger name.



Note: Properties in the logging configuration are not system properties. You cannot set them with the `-D` command-line option. Instead, place the logging properties into a file that you specify with `-Djava.util.logging.config.file=configFile`.

You can also specify the root level:

```
.level=WARNING
```



Caution: For historical reasons, some of the levels have different names in the platform logging API and the `java.util.logging` framework. You need to use the latter in the configuration file. **Table 7.1** shows the correspondences.

Table 7.1: Corresponding Levels for Platform Logging and the Java Logging Framework

Platform Logging	`java.util.logging`

ERROR	SEVERE
`WARNING`	`WARNING`
INFO	INFO
`DEBUG`	`FINE`
TRACE	FINER

Similar to package names, logger names are hierarchical. In fact, they are more hierarchical than packages. There is no semantic relationship between a package and its parent, but logger parents and children share certain properties. For example, if you turn off messages to the logger `"com.mycompany"`, then its child loggers are also deactivated.

As you will see in the next section, loggers don't actually send the messages to the console—that is the job of the handlers. Handlers also have levels. To see `DEBUG`/`FINE` messages on the console, you also need to set

```
java.util.logging.ConsoleHandler.level=FINE
```



Caution: The settings in the log manager configuration are not system properties. Starting a program with `-Dcom.mycompany.myapp.level=FINE` does not have any effect on the logger.

It is also possible to change logging levels in a running program by using the `jconsole` program. For details, see **https://www.oracle.com/technical-resources/articles/java/jconsole.html#LoggingControl**.



Note: As of Java 21, the system logger with name `java.lang.Runtime` logs calls to the `Runtime.exit` method (which `System.exit` calls to terminate the virtual machine). The log contains the stack trace, so you can tell how termination was requested. To receive the log message, set

```
java.lang.Runtime.level=FINE
```

in the log configuration.

7.5.5. Log Handlers

The backend of the `java.util.logging` API is based on *handlers*. The simplest handler is the `ConsoleHandler` that prints log records to the `System.err` stream.

Each handler has a parent. By default, the handler simply sends each record to the parent handler. The standard logging configuration uses a `ConsoleHandler` by default. Its default logging level is `INFO`.

To send log records elsewhere, add another handler. The `java.util.logging` API provides two handlers for this purpose: a `FileHandler` and a `SocketHandler`. The `SocketHandler` sends records to a specified host and port. Of greater interest is the `FileHandler` that collects records in a file.

To add a file handler, provide this entry in the logging properties:

```
handlers=java.util.logging.ConsoleHandler,java.util.logging.FileHandler
```

The records are sent to a file `javan.log` in the user's home directory, where `n` is a number to make the file unique. By default, the records are formatted in XML. A typical log record has the form

```
<record>
    <date>2025-12-04T09:53:34</date>
    <millis>1407146014072</millis>
    <sequence>1</sequence>
    <logger>com.mycompany.myapp</logger>
    <level>INFO</level>
    <class>com.horstmann.corejava.Employee</class>
    <method>read</method>
    <thread>10</thread>
    <message>Opening file staff.txt</message>
</record>
```

You can modify the default behavior of the file handler by setting various parameters in the logging properties (see **Table 7.2**).

You probably don't want to use the default log file name. Use a pattern such as `%h/myapp%u.log` (see **Table 7.3** for an explanation of the pattern variables). The `%u` in the file name pattern yields a unique copy of the log for each application run.

For long-running programs, it is a good idea to turn file rotation on. Log files are kept in a rotation sequence, such as `myapp.log.0`, `myapp.log.1`, `myapp.log.2`, and so on. Whenever a file exceeds the size limit, the oldest log is deleted, the other files are renamed, and a new file with generation number `0` is created.

Table 7.2: File Handler Configuration Parameters

Configuration Property	Description	Default
`java.util.logging.FileHandler.level`	The handler level.	`Level.ALL`

`java.util.logging.FileHandler.append`	When `true`, log records are appended to an existing file; otherwise, a new file is opened for each program run.	`false`
`java.util.logging.FileHandler.limit`	The approximate maximum number of bytes to write in a file before opening another (`0` = no limit).	`0` in the `FileHandler` class, `50000` in the default log manager configuration
`java.util.logging.FileHandler.pattern`	The file name pattern (see **Table 7.3**).	`%h/java%u.log`
`java.util.logging.FileHandler.count`	The number of logs in a rotation sequence.	`1` (no rotation)
`java.util.logging.FileHandler.filter`	The filter for filtering log records (see **Section 7.5.6**).	No filtering
`java.util.logging.FileHandler.encoding`	The character encoding.	The platform character encoding
`java.util.logging.FileHandler.formatter`	The formatter for each log record.	`java.util.logging.XMLFormatter`

Table 7.3: Log File Pattern Variables

Variable	Description
`%h`	The user's home directory (the `user.home` property).

`%t`	The system's temporary directory.
`%u`	A unique number.
`%g`	The generation number for rotated logs. A `.%g` suffix is used if rotation is specified and the pattern doesn't contain `%g`.
`%%`	The percent character.



Caution: When `java.util.logging.FileHandler.append` is `true`, and the log format is XML, then a new XML header is emitted when appending to an existing log file. The result is not valid XML.

7.5.6. Filters and Formatters

Besides filtering by logging levels, each handler can have an additional filter that implements the `Filter` interface, a functional interface with a method

```
boolean isLoggable(LogRecord record)
```

To install a filter into a handler, add an entry such as the following into the logging configuration:

```
java.util.logging.ConsoleHandler.filter=com.mycompany.myapp.MyFilter
```

The `ConsoleHandler` and `FileHandler` classes emit the log records in text and XML formats. However, you can define your own formats as well. Extend the `java.util.logging.Formatter` class and override the method

```
String format(LogRecord record)
```

Format the record in any way you like and return the resulting string. In your `format` method, you can get information about the `LogRecord` by calling one of the methods in the API notes.

In your `format` method, you may want to call the method

```
String formatMessage(LogRecord record)
```

That method formats the message part of the record, looking up the message key in a resource bundle and substituting message format parameters.

Many file formats (such as XML) require head and tail parts that surround the formatted records. To achieve this, override the methods

```
String getHead(Handler h)
String getTail(Handler h)
```

Finally, set the formatter in the logging configuration:

```
java.util.logging.FileHandler.formatter=com.mycompany.myapp.MyFormatter
```

7.5.7. A Logging Recipe

With so many options for logging, it is easy to lose track of the fundamentals. The following recipe summarizes the most common operations.

1. For a simple application, choose a single logger. It is a good idea to give the logger the same name as your main application package, such as `com.mycompany.myprog`. You can always get the logger by calling

   ```
   System.Logger logger = System.getLogger("com.mycompany.myprog");
   ```

   For convenience, you may want to add static fields

   ```
   private static final System.Logger logger
      = System.getLogger("com.mycompany.myprog");
   ```

   to classes with a lot of logging activity.
2. The default logging configuration of the `java.util.logging` backend logs all messages of level `INFO` or higher to the console. To customize, prepare a file `logging.properties` with entries such as the following:

   ```
   handlers=java.util.logging.ConsoleHandler,java.util.logging.FileHandler
   com.mycompany.myapp.level=FINER
   java.util.logging.ConsoleHandler.level=FINE
   ```

   Then start your app with

   ```
   java -Djava.util.logging.config.file=logging.properties com.mycompany.myapp.Main
   ```

3. Now you are ready to log to your heart's content. Whenever you are tempted to print a message to the console, use a logger instead:

   ```
   logger.log(System.Logger.Level.TRACE, "File open dialog canceled");
   ```

   It is also a good idea to log unexpected exceptions. For example:

```
try {
    . . .
}
catch (SomeException e) {
    logger.log(System.Logger.Level.WARNING, string describing context, e);
}
```

Listing 7.2 puts this recipe to use with an added twist: Logging messages are also displayed in a log window, thanks to a handler whose code is in **Listing 7.3**.

Run the program as

```
java -Djava.util.logging.config.file=v1ch07/logging/logging.properties \
    v1ch07.logging.LoggingImageViewer
```

Listing 7.2 v1ch07/logging/LoggingImageViewer.java

```
1  package v1ch07.logging;
2
3  import module java.base;
4  import module java.desktop;
5  import javax.swing.filechooser.FileFilter;
6
7  import static java.lang.System.Logger.Level.*;
8
9  /**
10  * A modification of the image viewer program that logs various events. Run as java
11  * -Djava.util.logging.config.file=logging.properties logging.LoggingImageViewer
12  */
13 class LoggingImageViewer {
14     void main() {
15         EventQueue.invokeLater(() -> {
16             var frame = new ImageViewerFrame();
17             frame.setTitle("LoggingImageViewer");
18             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19             System.Logger logger = System.getLogger("com.horstmann.corejava");
20             logger.log(INFO, "Showing frame");
21             frame.setVisible(true);
22         });
23     }
24 }
25
26 /**
27  * The frame that shows the image.
28  */
29 class ImageViewerFrame extends JFrame {
30     private static final int DEFAULT_WIDTH = 300;
31     private static final int DEFAULT_HEIGHT = 400;
32
33     private JLabel label;
34     private static System.Logger logger = System.getLogger("com.horstmann.corejava")
35
36     public ImageViewerFrame() {
```

```
        logger.log(TRACE, "Entering ImageViewerFrame()");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // set up menu bar
        var menuBar = new JMenuBar();
        setJMenuBar(menuBar);

        var menu = new JMenu("File");
        menuBar.add(menu);

        var openItem = new JMenuItem("Open");
        menu.add(openItem);
        openItem.addActionListener(new FileOpenListener());

        var exitItem = new JMenuItem("Exit");
        menu.add(exitItem);
        exitItem.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent event) {
                logger.log(INFO, "Exiting.");
                System.exit(0);
            }
        });

        // use a label to display the images
        label = new JLabel();
        add(label);
        logger.log(TRACE, "Exiting ImageViewerFrame()");
    }

    private class FileOpenListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            logger.log(TRACE, "Entering ImageViewerFrame.FileOpenListener.actionPerf
                    event);

            // set up file chooser
            var chooser = new JFileChooser();
            chooser.setCurrentDirectory(new File("."));

            // accept all files ending with .gif
            chooser.setFileFilter(new FileFilter() {
                public boolean accept(File f) {
                    return f.getName().toLowerCase().endsWith(".gif") || f.isDirecto
                }

                public String getDescription() {
                    return "GIF Images";
                }
            });

            // show file chooser dialog
            int r = chooser.showOpenDialog(ImageViewerFrame.this);

            // if image file accepted, set it as icon of the label
            if (r == JFileChooser.APPROVE_OPTION) {
                String name = chooser.getSelectedFile().getPath();
                logger.log(DEBUG, "Reading file %s", name);
```

```
93                  label.setIcon(new ImageIcon(name));
94              }
95              else
96                  logger.log(DEBUG, "File open dialog canceled.");
97              logger.log(TRACE, "Exiting ImageViewerFrame.FileOpenListener.actionPerfo
98          }
99      }
100 }
```

Listing 7.3 v1ch07/logging/WindowHandler.java

```
1  package v1ch07.logging;
2
3  import module java.base;
4  import module java.desktop;
5  import module java.logging;
6
7  /**
8   * A handler for displaying log records in a window.
9   */
10 public class WindowHandler extends StreamHandler {
11     private JFrame frame;
12
13     public WindowHandler() {
14         frame = new JFrame();
15         var output = new JTextArea();
16         output.setEditable(false);
17         frame.setSize(200, 200);
18         frame.add(new JScrollPane(output));
19         frame.setFocusableWindowState(false);
20         frame.setVisible(true);
21         setOutputStream(new OutputStream() {
22             public void write(int b) {
23             } // not called
24
25             public void write(byte[] b, int off, int len) {
26                 output.append(new String(b, off, len));
27             }
28         });
29     }
30
31     public void publish(LogRecord record) {
32         if (!frame.isVisible()) return;
33         super.publish(record);
34         flush();
35     }
36 }
```

java.lang.System.Logger **9**

- `String getName()`
  returns the name of this logger.

- `boolean isLoggable(System.Logger.Level level)`
  returns `true` if this logger processes logs at the given level.
- `void log(System.Logger.Level level, String msg)`
- `void log(System.Logger.Level level, String msg, Throwable thrown)`
- `void log(System.Logger.Level level, Object obj)`
  log the given message and any provided `Throwable`, or `obj.toString`.
- `void log(System.Logger.Level level, Supplier<String> msgSupplier)`
- `void log(System.Logger.Level level, Supplier<String> msgSupplier, Throwable thrown)`
  If this logger processes logs at the given level, invokes the supplier and logs the result and any provided `Throwable`.
- `void log(System.Logger.Level level, String format, Object... params)`
  logs the formatted message with the given parameters.
- `void log(System.Logger.Level level, ResourceBundle bundle, String format, Object... params)`
  If `bundle` is `null`, uses the `format` string directly, otherwise looks it up using `format` as key in the bundle. Then uses the format string as a `MessageFormat` with the given parameters and logs the result.
- `void log(System.Logger.Level level, ResourceBundle bundle, String msg, Throwable thrown)`
  If `bundle` is `null`, logs `msg`, otherwise the string with `msg` as key in the bundle. Then logs `thrown` if not `null`.

`java.util.logging.Handler` 1.4

- `abstract void publish(LogRecord record)`
  sends the record to the intended destination.
- `abstract void flush()`
  flushes any buffered data.
- `abstract void close()`
  flushes any buffered data and releases all associated resources.
- `Filter getFilter()`
- `void setFilter(Filter f)`
  get and set the filter of this handler.
- `Formatter getFormatter()`
- `void setFormatter(Formatter f)`
  get and set the formatter of this handler.
- `Level getLevel()`
- `void setLevel(Level l)`
  get and set the level of this handler.

`java.util.logging.ConsoleHandler` 1.4

- `ConsoleHandler()`
  constructs a new console handler.

`java.util.logging.FileHandler` 1.4

- `FileHandler(String pattern)`
- `FileHandler(String pattern, boolean append)`
- `FileHandler(String pattern, int limit, int count)`
- `FileHandler(String pattern, int limit, int count, boolean append)`
- `FileHandler(String pattern, long limit, int count, boolean append)` 9

  construct a file handler. See for the pattern format. `limit` is the approximate maximum number of bytes before a new log file is opened. `count` is the number of files in a rotation sequence. If `append` is `true`, records should be appended to an existing log file.

`java.util.logging.LogRecord` 1.4

- `Level getLevel()`

  gets the logging level of this record.
- `String getLoggerName()`

  gets the name of the logger that is logging this record.
- `ResourceBundle getResourceBundle()`
- `String getResourceBundleName()`

  get the resource bundle, or its name, to be used for localizing the message, or `null` if none is provided.
- `String getMessage()`

  gets the "raw" message before localization or formatting.
- `Object[] getParameters()`

  gets the parameter objects, or `null` if none is provided.
- `Throwable getThrown()`

  gets the thrown object, or `null` if none is provided.
- `String getSourceClassName()`
- `String getSourceMethodName()`

  get the location of the code that logged this record. This information may be supplied by the logging code or automatically inferred from the runtime stack. It might be inaccurate if the logging code supplied the wrong value or if the running code was optimized so that the exact location cannot be inferred.
- `long getMillis()`

  gets the creation time, in milliseconds since 1970.
- `Instant getInstant()` 9

  gets the creation time as a `java.time.Instant` (see **Chapter 6 of Volume II**).
- `long getSequenceNumber()`

  gets the unique sequence number of this record.
- `long getLongThreadID()` 16

  gets the unique ID for the thread in which this record was created. (The `getThreadID` method returns `int` IDs. It is now deprecated because a

long-running program might generate more than `Integer.MAX_VALUE` thread IDs.)

java.util.logging.Filter **1.4**

- `boolean isLoggable(LogRecord record)`
  returns `true` if the given log record should be logged.

`java.util.logging.Formatter` **1.4**

- `abstract String format(LogRecord record)`
  returns the string that results from formatting the given log record.
- `String getHead(Handler h)`
- `String getTail(Handler h)`
  return the strings that should appear at the head and tail of the document containing the log records. The `Formatter` superclass defines these methods to return the empty string; override them if necessary.
- `String formatMessage(LogRecord record)`
  returns the localized and formatted message part of the log record.

7.6. Debugging Tips

Suppose you wrote your program and made it bulletproof by catching and properly handling all exceptions. Then you run it, and it does not work right. Now what? (If you never have this problem, you can skip the remainder of this chapter.)

Of course, it is best if you have a convenient and powerful debugger. Debuggers are available as a part of professional development environments such as Eclipse, IntelliJ, and NetBeans. In this section, I offer a number of tips that may be worth trying before you launch the debugger.

1. You can print or log the value of any variable with code like this:

   ```
   IO.println("x=" + x);
   ```

   or

   ```
   logger.log(DEBUG, "x=" + x);
   ```

   If `x` is a number, it is converted to its string equivalent. If `x` is an object, Java calls its `toString` method. It can also be useful to log the state of the `this` object.

   ```
   logger.log(DEBUG, "this=" + this);
   ```

Most of the classes in the Java library are very conscientious about overriding the `toString` method to give you useful information about the class. This is a real boon for debugging. You should make the same effort in your classes.

2. You can put a separate `main` method in each class. Inside it, put code that demonstrates how the class should be used.

```
public class MyClass {
    methods and fields
    . . .
    void main() {
        demo code
    }
}
```

Make a few objects, call methods, and show that each of them does the right thing.

3. For more professional unit testing, you should check out JUnit from **https://junit.org**. JUnit is a very popular unit testing framework that makes it easy to organize suites of test cases. Run the tests whenever you make changes to a class, and add another test case whenever you find a bug.

4. A *logging proxy* is an object of a subclass that intercepts method calls, logs them, and then calls the original method. For example, if you have trouble with the `nextDouble` method of the `Random` class, you can create a proxy object as an instance of an anonymous subclass:

```
var generator = new Random()
   {
      public double nextDouble()
      {
         double result = super.nextDouble();
         logger.log(DEBUG, "nextDouble: " + result);
         return result;
      }
   };
```

Whenever the `nextDouble` method is called, a log message is generated. Note that this only works if the class is not `final`. With interfaces, one can implement a proxy that logs all methods, as shown in **Chapter 6**.

5. You can get a stack trace from any exception object with the `printStackTrace` method in the `Throwable` class. The following code catches any exception, prints the exception object and the stack trace, and rethrows the exception so it can find its intended handler.

```
try {
    . . .
}
catch (Throwable t) {
```

```
    t.printStackTrace();
    throw t;
}
```

You don't even need to catch an exception to generate a stack trace. Simply insert the statement

```
Thread.dumpStack();
```

anywhere into your code to get a stack trace.

6. Normally, the stack trace is displayed on `System.err.` If you want to log or display the stack trace, here is how you can capture it into a string:

```
var out = new StringWriter();
new Throwable().printStackTrace(new PrintWriter(out));
String description = out.toString();
```

7. It is often handy to trap program errors in a file. However, errors are sent to `System.err`, not the standard console output. Therefore, you cannot simply trap them by running

```
java MyProgram > errors.txt
```

Instead, capture the error stream as

```
java MyProgram 2> errors.txt
```

To capture both `System.err` and console output in the same file, use

```
java MyProgram 1> errors.txt 2>&1
```

This works in `bash` and the Windows shell.

8. Having the stack traces of uncaught exceptions show up in `System.err` is not ideal. These messages are confusing to end users if they happen to see them, and they are not available for diagnostic purposes when you need them. A better approach is to log them. You can change the handler for uncaught exceptions with the static `Thread.setDefaultUncaughtExceptionHandler` method:

```
Thread.setDefaultUncaughtExceptionHandler((Thread t, Throwable e) ->
      logger.log(TRACE, "Uncaught exception in " + t, e));
```

9. To watch class loading, launch the Java virtual machine with the `-verbose` flag. You will get a printout such as the following:

```
[0.012s][info][class,load] opened: /opt/jdk-21.0.1/lib/modules
[0.034s][info][class,load] java.lang.Object source: jrt:/java.base
[0.035s][info][class,load] java.io.Serializable source: jrt:/java.base
[0.035s][info][class,load] java.lang.Comparable source: jrt:/java.base
[0.035s][info][class,load] java.lang.CharSequence source: jrt:/java.base
[0.035s][info][class,load] java.lang.String source: jrt:/java.base
[0.036s][info][class,load] java.lang.reflect.AnnotatedElement source: jrt:/java.base
[0.036s][info][class,load] java.lang.reflect.GenericDeclaration source: jrt:/java.base
[0.036s][info][class,load] java.lang.reflect.Type source: jrt:/java.base
[0.036s][info][class,load] java.lang.Class source: jrt:/java.base
[0.036s][info][class,load] java.lang.Cloneable source: jrt:/java.base
[0.037s][info][class,load] java.lang.ClassLoader source: jrt:/java.base
[0.037s][info][class,load] java.lang.System source: jrt:/java.base
[0.037s][info][class,load] java.lang.Throwable source: jrt:/java.base
[0.037s][info][class,load] java.lang.Error source: jrt:/java.base
[0.037s][info][class,load] java.lang.ThreadDeath source: jrt:/java.base
[0.037s][info][class,load] java.lang.Exception source: jrt:/java.base
[0.037s][info][class,load] java.lang.RuntimeException source: jrt:/java.base
[0.038s][info][class,load] java.lang.SecurityManager source: jrt:/java.base
. . .
```

   This can occasionally be helpful to diagnose class path problems.

10. The `-Xlint` option tells the compiler to spot common code problems. For example, if you compile with the command

```
javac -Xlint sourceFiles
```

    the compiler will report missing `break` statements in `switch` statements and many other categories of warnings. (The term “lint” originally described a tool for locating potential problems in C programs, but is now generically applied to any tools that flag constructs that are questionable but not illegal.)
    You will get messages such as

```
warning: [fallthrough] possible fall-through into case
```

    The string in square brackets identifies the warning category. You can enable and disable each category. Since most of them are quite useful, it seems best to leave them all in place and disable only those that you don’t care about, like this:

```
javac -Xlint:all,-fallthrough,-serial sourceFiles
```

    You get a list of all warnings from the command

```
javac --help-lint
```

11. The Java VM has support for *monitoring and management* of Java applications, allowing the installation of agents in the virtual machine that track memory consumption, thread usage, class loading, and so on. This feature is particularly important for large and long-running Java programs. As a demonstration of these capabilities, the JDK ships with a graphical tool called `jconsole` that displays statistics about the performance of a virtual machine (see **Figure 7.2**). Start your program, then start `jconsole` and pick your program from the list of running Java programs.

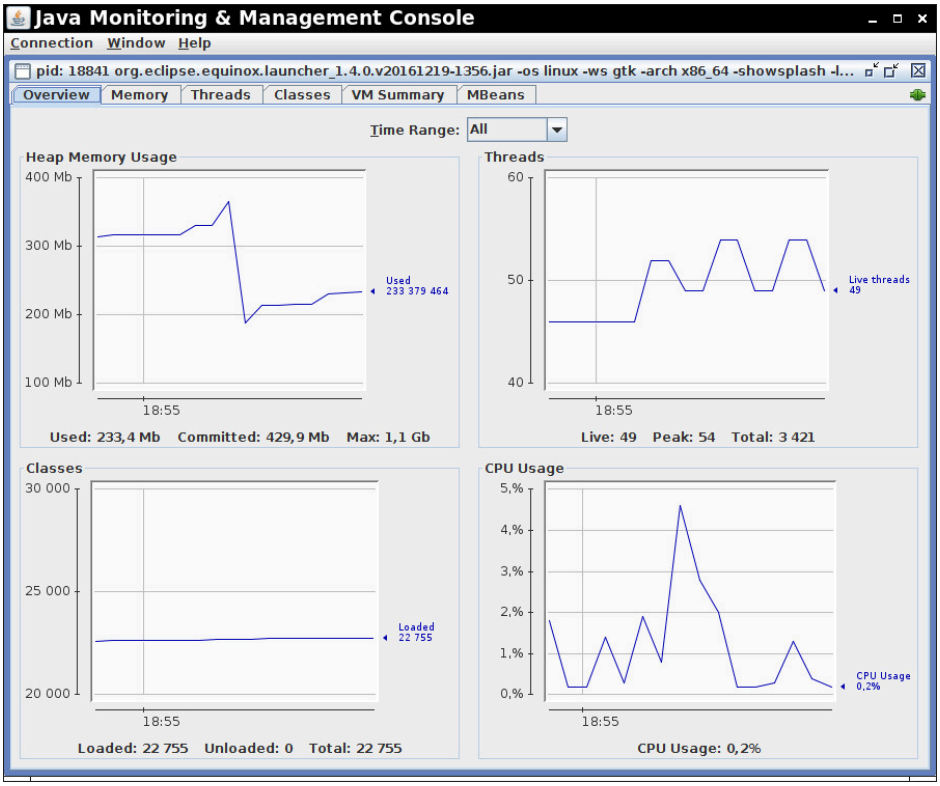


Figure 7.2: The `jconsole` program

This program gives you a wealth of information about your running program. See **https://www.oracle.com/technical-resources/articles/java/jconsole.html** for more information.

12. Java Flight Recorder is an instrumentation technology of the OpenJDK virtual machine that collects diagnostic and profiling data. You can use the open-source VisualVM (**https://visualvm.github.io**) and Java Mission Control (**https://adoptium.net/jmc**) to collect and view flight recording data. These tools also do everything that you can do with `jconsole`. See **https://github.com/thegreystone/jmc-tutorial** for a comprehensive tutorial for the latter.

13. The JDK comes with a number of command-line tools with overlapping use cases for monitoring the virtual machine (`jcmd`, `jhsdb`, `jinfo`, `jmap`, `jps`, `jstat`). The `jcmd` tool is a good starting point since it covers several of those use cases. Run

```
jcmd
```

to get a list of all Java processes on your computer, with process ID and the name of the main class.
To list system properties or VM flags, run

```
jcmd pidOrMainClass VM.system_properties
jcmd pidOrMainClass VM.flags
```

You can configure the virtual machine log:

```
jcmd pidOrMainClass VM.log output=gc.log what=gc=trace decorators=uptime,tid
```

To get a thread dump, run

```
jcmd pidOrMainClass Thread.print
```

or

```
jcmd pidOrMainClass Thread.dump_to_file -overwrite -format=json dump.json
```

To start Java Flight Recorder, run

```
jcmd pidOrMainClass JFR.start filename=filename
```

To stop recording, run

```
jcmd pidOrMainClass JFR.stop name=1
```

To get a list of all available commands, run

```
jcmd pidOrMainClass
```

To get help for a particular command, run

```
jcmd pidOrMainClass help commandName
```

This chapter introduced you to exception handling and logging. You also saw useful hints for testing and debugging. The next two chapters cover generic programming and its most important application: the Java collections framework.

Previous chapter
< 6. Interfaces, Lambda Expressions, and Inner Classes

Next chapter
8. Generic Programming >